Restaurant for Gophers

PCLT Project



Authors:

Daniel Eugénio 59797

Renato Viola 60665

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

In this report, we discuss the various design choices made during the implementation of a simulation of a restaurant with concurrent interactions between customers, chefs and waiters.

Correctness

To represent the assignment's scenario adequately, we began by modelling each one of its entities in their own goroutine and defined a new datatype representing an *Order.* As, in this case, a customer's purpose is to come to the restaurant and order one single item to have delivered, it made sense to us that each *Order* is identified by the ID of the customer who placed it (*id*), the rest of its attributes being: the description of ordered dish itself (*dish*), a synchronous boolean channel signaling the food's delivery and correspondent acknowledgment (*deliveryCh*) and finally, the owning customer's context (*ctxt*).

To ensure correctness in our system, we had to coordinate the communication between the separate and independent goroutines (the entities) with the use of not only Go's channels but also contexts, both of which are safe for concurrent use. Not only that, but the *Order* objects that are transported through these channels are treated as immutable after their creation.

In order to make sure that events weren't occurring out of place, we used buffered channels to synchronize the goroutines with one another and even modeled them in such a way that they could only use the channels unidirectionally, in a way "prohibiting" a routine from using the channels in a way it isn't supposed to. This way, we've ensured that chefs only prepare orders that have been placed and waiters only deliver orders that are ready to be served.

The channels used in these cases are buffered because nothing would be gained from having these routines waiting on each other, if proper sequence of operations is already achieved. We tried to choose reasonable buffer sizes that could accommodate small bursts of orders without blocking.

To signal the delivery of an order from a waiter to a customer and simulate the customer's acknowledgement of having received it before the waiter can move on to the next order, we actually used an unbuffered channel (*deliveryCh*), precisely because of its blocking behavior. Here, what we really want is for the waiter to be blocked until the customer is prepared to accept the order, forcing him to wait until then before proceeding to the next order.

A customer's willingness to wait is simulated through a timeout, this is done by having each customer be assigned a context with a timeout in its goroutine. This context is part of the order's composition so both a chef or a waiter can check for its validity. If the order is not delivered within this timeframe, the customer leaves and the context is canceled.

Neither chefs nor waiters will begin work on an order that's already been canceled, they'll just skip it and move on to the next order instead, as to not waste unnecessary time and resources. To this end, the first thing any of these workers does upon picking up an

order is checking the validity of its context, only then proceeding to process the order if it is indeed “live”.

This check only happens once at the beginning of the routines, for example a chef needn’t be concerned about the liveness of an order he already prepared or started preparing, for the waiter will handle that verification.

Component Management

Role organization in the system

Customers: Customers select a dish, create an order, which is sent to the placedOrders channel; They then wait for their order on the deliveryCh channel or leave if their context times out.

Chefs: They receive orders from the placedOrders channel, check if the customer is still present, and prepare the dish; Prepared orders are sent to the readyOrders channel for waiters to pick up.

Waiters: They receive ready orders from the readyOrders channel, check the customer's context, and deliver the order via deliveryCh; Waiters wait until the customer receives the order before proceeding to the next one.

Communication Between Components

In our implementation, we decided to use three channels, two of which are buffered to avoid blocking the execution of routines and a synchronous channel in order to guarantee the waiter waits for the acknowledgement from the customer. Their meaning is described as follows:

placedOrders: Buffered channel for customers to place orders, and where the chefs go fetch them for preparation. Buffer size of 10 seemed reasonable enough to handle the initial wave of incoming customers and other small customer bursts without much blocking.

readyOrders: Buffered channel for chefs to pass prepared orders to waiters. Buffer size of 5 seems more than enough to have the channel act as a queue for all the ready orders, as there are only three chefs in the system who do take some time preparing orders, and a waiter delivers a ready order instantly.

deliveryCh: Unbuffered channel for waiters to deliver orders to customers, ensuring waiter doesn’t move on until customer acknowledgement.

Handling customer timeouts

Contexts were used to handle customer timeouts, simulating a certain amount of time each customer was willing to wait for their food. Every customer has their own available time that falls inside a specific range. Chefs and waiters check the context before processing orders to ensure the customer is still present, skipping the order if it is no longer valid.

Additional Comments

Color-coded logs for easier debugging

When testing our implementation by running the program, the console would obviously get cluttered with information regarding several different events. This made it quite difficult to keep track of what was going on and keep track of the sequence of events taking place, so we decided to color code different logs as follows:

1. Logs signaling orders delivered by a waiter to a customer and the resultant acknowledgement by the client where assigned a green color.
2. Logs signaling that a client left, as well as when a chef or a waiter discard a dish are marked in red.
3. Logs signaling that a chef has started preparing an order are encoded in blue.
4. Logs signaling that a chef finished preparing an order are marked in yellow.

This was done so that we could easily compare events of the same nature and assessing the correct order of events is being followed. To achieve this result we were heavily inspired by the following publication by Stephanie You: Adding Color to Terminal Output in Go

Customer arrival rate varies with time

In our system, the average time a customer takes to enter the system increases along the program's execution time.

We thought it would contribute an interesting dynamic to the simulation, as it sort of simulates a rush hour in a real restaurant and also periods of lesser affluence. We did this to have a relatively large flow of customers and orders in the beginning, almost as if limit-testing the restaurant.

During the initial moments of execution, with many customers, several timeouts are to be expected as there are not enough chefs to prepare all the orders being placed and especially within the time constraints defined by the short fuse of some customers.

As the program goes on, and less and less customers are coming in, it becomes clear that the system can handle the load of orders much more comfortably, as most of the orders now get to be successfully delivered to the customers, and timeouts are a much rarer occasion.